

Módulo 6: Infraestructura y Deploy

T16: Base de Datos Segura

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Mínimo privilegio, backups cifrados y el puerto de BD nunca expuesto*

Objetivos

- Aplicar el principio de mínimo privilegio en la configuración de la BD
- Entender por qué el puerto de BD nunca debe estar expuesto al exterior
- Configurar backups cifrados
- Auditar las queries lentas y permisos con herramientas básicas

Principio de mínimo privilegio en BD

✗ Mal:

```
-- Un solo usuario con TODOS los permisos  
GRANT ALL PRIVILEGES ON *.* TO 'app';
```

✓ Bien:

```
-- Usuario de app: solo lo que necesita  
GRANT SELECT, INSERT, UPDATE ON vulndb.* TO 'app_user';  
-- Usuario de admin: separado  
GRANT ALL ON vulndb.* TO 'admin_user';  
-- Usuario de backup: solo lectura  
GRANT SELECT ON vulndb.* TO 'backup_user';
```

⚠ **VulnApp con SQLite** no tiene sistema de roles, pero el principio aplica a cualquier BD en producción (PostgreSQL, MySQL, etc.).

El puerto de BD NUNCA expuesto al exterior

```
# docker-compose.yml
services:
  db:
    image: postgres:16
    # ✗ NUNCA hacer esto en producción:
    # ports:
    #   - "5432:5432" # ← accesible desde Internet

    # ✔ Solo accesible desde la red interna de Docker:
    expose:
      - "5432"
    networks:
      - internal

  app:
    networks:
      - internal
      - public

networks:
  internal: # solo app <-> db
  public:   # solo app <-> internet
```

Backups cifrados

```
# 1. Dump de la BD
pg_dump -U app_user vulndb > backup.sql

# 2. Cifrar con GPG
gpg --symmetric --cipher-algo AES256 backup.sql
# Genera: backup.sql.gpg

# 3. Borrar el dump en claro
rm backup.sql

# 4. Subir el cifrado a almacenamiento externo
az storage blob upload --file backup.sql.gpg --container backups
```

✅ **Regla 3-2-1:** 3 copias, 2 medios distintos, 1 offsite. Todas cifradas.

Checklist de seguridad de BD

Aspecto	Inseguro	Seguro
Puerto	Expuesto al exterior	Solo red interna
Usuario	ALL PRIVILEGES	Mínimo privilegio por rol
Contraseña	Corta / en código	Larga, aleatoria, en Key Vault
Backups	Sin cifrar, sin probar	Cifrados + restore test periódico
Conexión	Sin TLS	TLS requerido (<code>sslmode=require</code>)
Logs	Sin auditoría	Query logging + alertas anomalías

Cifrado at-rest y en tránsito

Capa	Qué protege	Cómo
At-rest	Datos en disco (robos físicos, backups filtrados)	TDE, SQLCipher, LUKS
En tránsito	Datos entre app y BD	<code>sslmode=require</code> en connection string
A nivel de campo	Columnas sensibles (PII, tarjetas)	Cifrado aplicativo (AES-256-GCM)

```
// Conexión PostgreSQL con TLS obligatorio
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: true, ca: fs.readFileSync('/path/to/ca.pem') }
});
```

✓ En Azure/AWS, los servicios gestionados (Azure SQL, RDS) cifran at-rest por defecto.

Inyección a nivel de BD — stored procedures seguras

```
-- ❌ Procedure vulnerable (concatenación)
CREATE FUNCTION search_user(query TEXT) RETURNS SETOF users AS $$
BEGIN
    RETURN QUERY EXECUTE 'SELECT * FROM users WHERE name LIKE '%' || query || '%'';
END;
$$ LANGUAGE plpgsql;

-- ✅ Procedure segura (parámetro)
CREATE FUNCTION search_user(query TEXT) RETURNS SETOF users AS $$
BEGIN
    RETURN QUERY SELECT * FROM users WHERE name LIKE '%' || quote_literal(query) || '%';
END;
$$ LANGUAGE plpgsql;
```

⚠️ La inyección SQL también puede ocurrir **dentro** de stored procedures si concatenan input.

SQLite en VulnApp – particularidades

```
// El archivo vuln.db es la BD completa  
const DB_PATH = path.join(__dirname, '../vuln.db');
```

Riesgos específicos de SQLite en producción:

- Archivo legible por cualquier proceso con acceso al filesystem
- Sin sistema de usuarios/roles nativo
- Sin cifrado por defecto (usar SQLCipher para cifrado at-rest)
- Sin conexiones remotas (ventaja en este caso)

💡 SQLite es adecuado para desarrollo/prototipo. En producción real: PostgreSQL o similar.

Lab T16: Asegurando la base de datos de VulnApp

Objetivo: configurar la BD de VulnApp con mínimo privilegio

Setup: `cd VulnApp`

1. Inspecciona la configuración actual de la BD en `src/db/database.ts`
2. Verifica si hay permisos o accesos innecesarios habilitados
3. Asegura que la BD no es accesible desde fuera de localhost en Docker Compose
4. Implementa backups automáticos con `sqlite3 vuln.db .dump > backup.sql`